Booklet no:_____

ID: _____

Algorithms – Final Exam

6.2.23 – Term A

Prof. Tami Tamir

It's forbiden to answer on the other side of the page! Do not use marker!

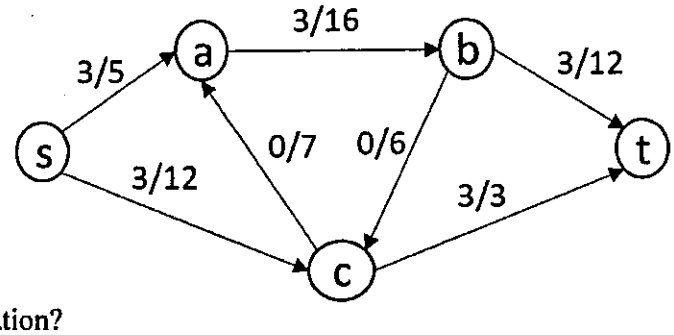
- Exam length: 3 hours
- Allowed open material: Attached fact-sheet + one reference paper.
- You can cite and rely (without providing a proof) on theorems and claims that were proven in class or in the homework.
- In every question or section marked by ♣, you can write "I don't know" and get one quarter of the score (rounded down).

GOOD LUCK!

♣ Question 1 (20 pts.)

♣ **1.1** (12 pts.) The figure presents a flow network with a legal flow. Every edge $e \in E$ is labeled by f(e)/c(e) . It is known that the given flow is a result of **two** iterations of the Ford & Fulkerson algorithm.

No need to justify your answers in this section.



1. What was the augmenting path in the first iteration?

2. What was the augmenting path in the second iteration?

3. Suggest an augmenting path for the next (third) iteration:

4. What is the value of the maximum flow in the network?

5. Write all the sets of vertices that define a min-cut in the network:

6. Write all the edges satisfying the following condition: If the capacity of this edge alone is increased by 1, then the value of the max-flow in the network is increased by 1:

1.2 (2*4=8 points). Let G=(V,E) be a flow network. For a given edge $e_1 \in E$, it is known that if the capacity of e_1 is increased by 4, then the value of the max-flow in the network increases by 2.

For each of the following descriptions (separately), determine whether it is possible. If yes - justify with an example (draw the network before and after the capacity change, and specify why the description holds). If not - explain why.

♣ a. The edge e_1 belongs to a min-cut of G after the capacity change.

♣ b. After the capacity change, there is a max-flow in which the edge e_1 is saturated.

Question 2 (3*4=12 pts.)

Let G=(V,E) be a **simple directed** graph over $|V| \geq 2$ vertices.

Reminder: a **root** in a directed graph is a vertex from which there is a path to each vertex in the graph.

Prove or disprove each of the following three claims (separately):

♣ **2.1** If G has at least two roots, then $|E| \geq |V|$.

♣ **2.2** If $|E| \geq 2|V|$, then G has at least one root.

♣ **2.3** If G is strongly connected and $|V|=|E|$, then there is a Hamiltonian cycle in G.

Question 3 (18 pts.)

Eran is planning a long trip that will last N days. Eran will visit B different countries during the trip. He needs to decide how many days he will stay in each country. For each $1 \le j \le B$ and $1 \le k \le N$, a stay lasting k days in the j-th country will cost Eran C_{jk} Shekels. Eran wishes to stay in each country at least three days.

Suggest an algorithm based on **dynamic programming** that receives as input N, B, and the values C_{jk}, and calculates the minimum cost of the entire trip. It can be assumed that $N \ge 3B$.

For each $1 \le j \le B$ and $1 \le d \le N$, denote by MinCost[j,d] the minimum cost of a trip lasting d days in the first j countries (and in each, at least three days).

3.1 (2 pts.) What is the expression describing the solution's value (no need to explain)?

♣ **3.2** (12 pts.) Write a dynamic programming formula for calculating MinCost[j,d]. Distinguish between the base and the general case. Explain the formula in words.

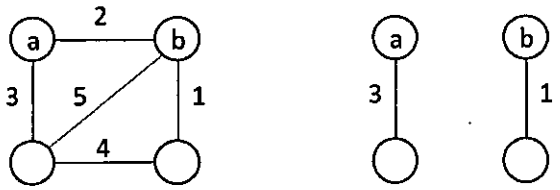
♣ **3.3** (4 pts.) Describe the DP-table, its size, and the order it is filled. Analyze the time-complexity of the algorithm as a function of B and N. Justify your answer.

♣ Question 4 (18 pts.)

Let G=(V,E) be a simple, connected, and undirected graph, with positive edge weights. A set of edges $E' \subseteq E$ is called an **almost-spanning-tree** of G if it has n-2 edges and includes no cycles. Note that an almost-spanning-tree consists of two connected components, each is a tree.

Given two vertices $a,b \in V$, **suggest an algorithm** for finding an almost-spanning-tree of G in which the vertices a and b belong to different connected components, and whose total edge weights is minimal. **Prove** the correctness of your algorithm, and **specify** its time complexity.

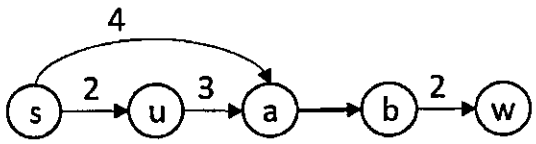
Example: A graph G (left), and an almost-spanning-tree of G in which a and b are separated and has minimum weight (right).



♣ Question 5 (18 pts.)

Given is a directed graph $G = (V, E)$, a vertex $s \in V$, and an edge $e_1 = (a, b) \in E$.
Let $c: E \setminus \{e_1\} \to \mathbb{R}^+$ be a positive cost function defined for all the graph edges, except for the edge e_1, which is not given a cost. In a path that includes the edge e_1, the cost of passing through e_1 is the same as the cost of **the first edge in the path**. Suggest an efficient algorithm that finds for each vertex $v \in V$ the cost of a cheapest path from s to v.
Describe the algorithm in words and formally. Justify its correctness shortly. No need to analyze its complexity.

Example: In the graph in the figure,
d(s,s)=0, d(s,u)=2, d(s,a)=4, d(s,b)=7, d(s,w)=9.
The price of the path s-u-a-b is 2+3+2=7. In this path, the cost of the edge (a,b) is 2 because the first edge in the path is (s,u). The cost of the path s-a-b is 4+4=8. In this path, the cost of (a,b) is 4.



Question 6 (14 pts.)

Along "The Greedy" street there are n restaurants. For each $1 \le i \le n$, the i-th restaurant is located at distance $x_i \ge 0$ from the beginning of the street (west-most point). The owners of the restaurants want to install wi-fi routers along the street. All the wi-fi routers are identical and have covering range $R \ge 0$. That is, a router located at distance p from the beginning of the street can serve users at restaurants in the interval [max(0,p-R), p+R]. The restaurant owners want to determine the locations of the routers so that each restaurant has at least one router that covers it, and the total number of routers required to cover all the restaurants on the street is minimal.

♣ 6.1 (4 points) Show by giving an example that the following algorithm is not optimal:
While there is a restaurant that is not covered yet, determine the next router at point p for which the number of restaurants that are not covered yet and are in the interval [max(0,p-R), p+R] is maximal.

♣ **6.2** (5 pts.) Suggest an optimal greedy algorithm for the problem. The algorithm should be efficient but there is no need to analyze its complexity or to explain its implementation.

♣ **6.3** (5 pts.) **State** and **prove** the corresponding greedy choice property.

Spare page – use if needed.

